

Introduction

Every experienced engineer has had this moment: you open a pull request, scroll through a class you've never seen before, and something about its shape feels *familiar*. You've seen this structure before — an object holding another object, calling one method on it, and somehow producing wildly different behavior depending on what got handed in. You can't quite name it, but you know it's a pattern.

That instinct is worth more than memorizing a glossary of pattern names. Interviewers know this too — when a coding interview drifts into system design or object-oriented design territory, they're rarely asking "define the Strategy pattern." They're handing you a messy requirement and watching whether you reach for the right shape of solution without being told which one to use.

This article walks through five of the most common design patterns — Strategy, Observer, State, Decorator, and Proxy — the way you'd actually encounter them: as a chunk of code you have to read cold and classify. Along the way we'll build the one habit that matters more than any dictionary of pattern definitions: learning to ask "who's actually in control here?"

By the end, you'll have a fast, repeatable way to tell these five patterns apart — including the one pairing that trips up almost every developer, even ones who've been writing object-oriented code for years.

Problem Overview

Design patterns exist because certain problems show up again and again in software, and over the decades the industry converged on a handful of reusable shapes for solving them. The catch is that most engineers learn patterns from static definitions — "Strategy encapsulates interchangeable algorithms" — and then fail to recognize the same pattern sitting right in front of them in a real codebase, because real code doesn't come labeled.

The five patterns in this article split into two families:

- **Behavioral patterns** (Strategy, Observer, State) are about how objects communicate and who controls the flow of decisions.
- **Structural patterns** (Decorator, Proxy) are about how objects are composed and wrapped around each other.

The goal isn't to recite definitions. It's to notice a small number of concrete "tells" in the code — who hands in the behavior, who's listening, who replaces what, and why something is being wrapped — and use those tells to name the pattern in seconds.

Example

Let's ground this in one running example: a payment processor.

Naive version (no pattern):

```
class PaymentProcessor:
    def pay(self, method, amount):
        if method == "credit_card":
            print(f"Charging ${amount} to credit card")
        elif method == "paypal":
            print(f"Charging ${amount} via PayPal")
        elif method == "crypto":
            print(f"Charging ${amount} via crypto wallet")
```

Every time the business adds a new payment method, you reopen this class and add another branch. This is the shape design patterns exist to eliminate.

Pattern version:

```
class PaymentProcessor:
    def __init__(self, strategy):
        self.strategy = strategy

    def pay(self, amount):
        self.strategy.pay(amount)
```

Now `PaymentProcessor` has no idea what kind of payment method it's dealing with. It just holds an object and calls `.pay()` on it. Swap in a `CreditCardStrategy` or a `PayPalStrategy` and the entire behavior changes — with zero edits to `PaymentProcessor` itself.

That contrast — hardcoded branching versus an object that delegates to whatever it's handed — is the seed for everything that follows.

Intuition

Here's how an experienced engineer actually arrives at these patterns, instead of pulling one off a shelf.

You almost never start by thinking "I should use Strategy here." You start by noticing a smell: a class is doing too much branching based on type, or a class is tightly coupled to something it shouldn't know about, or a piece of behavior needs to change at runtime without editing source code. The pattern is the *destination*, not the *starting point*.

The single most useful question, across all five patterns in this article, is:

Who is really in control?

- If an outside caller hands in the behavior as an object → **Strategy**.
- If one object is shouting an update to a crowd of listeners it doesn't know or care about → **Observer**.
- If an object hands control to *itself*, replacing its own internal state → **State**.
- If objects are wrapped in layers, each one adding new behavior → **Decorator**.
- If objects are wrapped in layers, but the wrapper's job is to *gatekeep* access rather than add behavior → **Proxy**.

Notice that Strategy and State look almost identical on paper — both involve a class holding a reference to another object and delegating a call to it. Same with Decorator and Proxy — both involve wrapping. The patterns aren't distinguished by structure alone. They're distinguished by *intent and direction of control*, which is exactly why textbook definitions fail people in practice: they describe the shape, not the intent.

Brute Force Solution

Before any pattern gets involved, most developers write the "brute force" version: a single class with conditional branches for every case, as shown in the naive `PaymentProcessor` above.

Idea: One class knows about every variant and picks behavior with `if / elif` or a lookup table.

Algorithm: 1. Receive a type indicator (a string, enum, or flag). 2. Branch on that indicator. 3. Execute the matching logic inline.

Advantages: - Fast to write for two or three cases. - Everything lives in one file — easy to read top to bottom when small.

Disadvantages: - Violates the Open/Closed Principle — every new case means editing existing, tested code. - Branches multiply combinatorially once more than one axis of variation exists (payment method × currency × region, for example). - Hard to unit test in isolation — you're always testing the whole class.

Time complexity: $O(n)$ per call in the worst case, where n is the number of branches, since it may need to check each condition in order. **Space complexity:** $O(1)$ beyond the branching class itself.

This isn't wrong for small, stable cases — but it's the version every pattern in this article exists to replace once change becomes frequent.

Optimal Solution

The "optimal" solution isn't one pattern — it's picking the *right* pattern for the *actual* axis of variation in your problem. Here's the decision table that captures all five:

Pattern	Question it answers	Who controls the swap	Structure
Strategy	Which algorithm should run?	The caller, from outside	Holds one interchangeable object
Observer	Who needs to know about this event?	The subject, broadcasting	Holds a list of listeners
State	What should this object do next?	The object itself	Holds one object that replaces itself
Decorator	What extra behavior should wrap this?	The caller, by nesting wrappers	Chain of wrappers adding behavior
Proxy	Should the real object even be touched?	The wrapper, gatekeeping	One wrapper controlling access

Walking through the five rounds from the video:

Round 1 — Strategy. A `PaymentProcessor` takes a strategy object in its constructor and calls one method on it. The caller decides which strategy to plug in. Think of a universal charger: the socket never changes, you just plug in a different adapter.

Round 2 — Observer. A `NewsFeed` keeps a list of subscribers and loops through all of them when a story publishes. Nobody asked it to run one specific algorithm — it's broadcasting to a crowd that only needs to expose an `update()` method.

Round 3 — State (the trap round). An `Order` holds a `status` object, and calling `next()` hands control to that status object, which decides what the new status should be. This looks identical to Strategy in code — an object delegating through another object. The difference is *who picks the replacement*. In Strategy, the caller injects it from outside. In State, the current state object swaps itself out from the inside.

Round 4 — Decorator. A `Coffee` object gets wrapped by `Milk`, then `Sugar`, then `Whip`. Each wrapper holds a reference to the object just inside it and adds its own cost before delegating down the

chain.

Round 5 — Proxy (the catch). A class wraps a real repository — structurally identical to Decorator. But it's not stacking new behavior on top. It's deciding *whether the real object even gets touched* — for example, checking a cache before hitting the database. Same shape as Decorator, completely different intent.

Python Code

Here's clean, runnable code for all five, using the payment/order/coffee/repository examples from the video.

```
from abc import ABC, abstractmethod

# --- Round 1: Strategy ---
class PaymentStrategy(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        ...

class CreditCardStrategy(PaymentStrategy):
    def pay(self, amount: float) -> None:
        print(f"Charged ${amount:.2f} to credit card")

class PayPalStrategy(PaymentStrategy):
    def pay(self, amount: float) -> None:
        print(f"Charged ${amount:.2f} via PayPal")

class PaymentProcessor:
    def __init__(self, strategy: PaymentStrategy):
        self.strategy = strategy

    def pay(self, amount: float) -> None:
        self.strategy.pay(amount)

# --- Round 2: Observer ---
class Subscriber(ABC):
    @abstractmethod
    def update(self, story: str) -> None:
        ...

class EmailSubscriber(Subscriber):
```

```

def __init__(self, name: str):
    self.name = name

def update(self, story: str) -> None:
    print(f"Emailing {self.name}: {story}")

class NewsFeed:
    def __init__(self):
        self.subscribers: list[Subscriber] = []

    def subscribe(self, subscriber: Subscriber) -> None:
        self.subscribers.append(subscriber)

    def publish(self, story: str) -> None:
        for subscriber in self.subscribers:
            subscriber.update(story)

# --- Round 3: State ---
class OrderState(ABC):
    @abstractmethod
    def next(self, order: "Order") -> None:
        ...

class Placed(OrderState):
    def next(self, order: "Order") -> None:
        print("Order shipped")
        order.state = Shipped()

class Shipped(OrderState):
    def next(self, order: "Order") -> None:
        print("Order delivered")
        order.state = Delivered()

class Delivered(OrderState):
    def next(self, order: "Order") -> None:
        print("Order already delivered")

class Order:
    def __init__(self):
        self.state: OrderState = Placed()

    def advance(self) -> None:
        self.state.next(self)

# --- Round 4: Decorator ---
class Beverage(ABC):

```

```

    @abstractmethod
    def cost(self) -> float:
        ...

class Coffee(Beverage):
    def cost(self) -> float:
        return 2.00

class Milk(Beverage):
    def __init__(self, wrapped: Beverage):
        self.wrapped = wrapped

    def cost(self) -> float:
        return self.wrapped.cost() + 0.50

class Sugar(Beverage):
    def __init__(self, wrapped: Beverage):
        self.wrapped = wrapped

    def cost(self) -> float:
        return self.wrapped.cost() + 0.25

# --- Round 5: Proxy ---
class Repository(ABC):
    @abstractmethod
    def get(self, key: str) -> str:
        ...

class RealRepository(Repository):
    def get(self, key: str) -> str:
        print(f"Hitting the database for {key}")
        return f"data-for-{key}"

class CachingProxy(Repository):
    def __init__(self, real_repo: Repository):
        self.real_repo = real_repo
        self.cache: dict[str, str] = {}

    def get(self, key: str) -> str:
        if key not in self.cache:
            self.cache[key] = self.real_repo.get(key)
        else:
            print(f"Cache hit for {key}")
        return self.cache[key]

```

Code Walkthrough

- `PaymentProcessor.__init__` takes any `PaymentStrategy` — the caller decides the algorithm at construction time, not the class itself.
- `NewsFeed.publish` iterates a list of subscribers it knows nothing about beyond the `update()` contract — this is the broadcast shape unique to Observer.
- `Order.advance` delegates to `self.state.next(self)`, and inside each `next()`, the state object reassigns `order.state` to a *new* state object — the object replacing itself, not being replaced from outside.
- `Milk / Sugar` each hold a `wrapped` reference and add their own cost before returning — this is the layering that defines Decorator.
- `CachingProxy.get` has the exact same shape as a decorator (`self.real_repo`, delegate the call) but the `if key not in self.cache` check is the tell: it's deciding *whether* to touch the real object at all.

Dry Run

Take the Decorator chain: `Whip(Sugar(Milk(Coffee())))`.

1. Call `.cost()` on the outermost `Whip` wrapper.
2. `Whip` calls `.cost()` on `Sugar`, planning to add its own fee after.
3. `Sugar` calls `.cost()` on `Milk`, planning to add `0.25` after.
4. `Milk` calls `.cost()` on `Coffee`, planning to add `0.50` after.
5. `Coffee.cost()` returns `2.00` directly — the base case.
6. `Milk` receives `2.00`, adds `0.50` → returns `2.50`.
7. `Sugar` receives `2.50`, adds `0.25` → returns `2.75`.
8. `Whip` receives `2.75`, adds its own fee (say `0.75`) → returns `3.50`.

The call cascades all the way down to the base object, then bubbles back up, picking up cost at every layer — that's the "runtime chain" the video points to.

Complexity Analysis

Pattern	Time	Space	Why
Strategy	$O(1)$ dispatch	$O(1)$ extra	One indirection call, no branching
Observer	$O(n)$ per publish	$O(n)$ subscribers	Must notify every listener
State	$O(1)$ per transition	$O(1)$ extra	One reassignment, no branching

Pattern	Time	Space	Why
Decorator	$O(k)$ per call, k = wrapper depth	$O(k)$ call stack	Each layer delegates once
Proxy (caching)	$O(1)$ amortized after first call	$O(n)$ cached keys	Cache lookup replaces repeated real calls

These are correct because none of the patterns introduce searching or branching proportional to the number of *variants* — that's precisely the cost the brute-force `if / elif` chain pays and these patterns eliminate.

Alternative Solutions

For **Strategy**, a plain function reference (or `functools.partial`) can replace a full class hierarchy in Python, since functions are first-class objects — often simpler than defining a `PaymentStrategy` ABC for two or three variants.

For **Observer**, Python's `dataclasses` combined with the standard library offers no built-in pub/sub, but frameworks (Django signals, `blinker`) implement Observer at scale — worth reaching for once you have many publishers and subscribers rather than hand-rolling the list.

For **Proxy**, Python's `functools.lru_cache` is effectively a caching proxy applied as a decorator on a function rather than a class — same intent, less boilerplate, when you don't need class-based state.

Edge Cases

- **Empty subscriber list (Observer):** `publish()` should simply do nothing — no error, no special-casing.
- **State with no valid transition:** `Delivered.next()` should either be a no-op or raise a clear exception rather than silently corrupting state.
- **Decorator with zero wrappers:** calling `.cost()` on a bare `Coffee()` should return the base cost with no errors — decorators must be optional, not required.
- **Proxy cache miss on every key:** if keys are always unique, the "cache" degrades to $O(n)$ space with no time savings — worth monitoring in production.
- **Strategy swapped mid-use:** if `self.strategy` is reassigned between calls, make sure nothing depends on the *previous* strategy's internal state.

Common Mistakes

1. **Confusing State and Strategy** because both hold a delegate object — check who assigns the replacement, not what the code looks like.
2. **Using Observer without unsubscribe logic**, causing memory leaks when subscribers should have been removed.
3. **Building a Decorator chain that isn't interchangeable** — if `Milk` requires a `Coffee` specifically instead of any `Beverage`, you've lost the pattern's flexibility.
4. **Treating Proxy as free** — a caching proxy needs cache invalidation, or it will silently serve stale data.
5. **Overusing Strategy for two static options** — if the behavior never changes at runtime, a simple `if` statement is clearer than a class hierarchy.

Interview Questions

- "How would you refactor this `if / elif` chain to avoid touching it every time a new case is added?"
- "What's the structural difference between Decorator and Proxy, and why does it matter in production code?"
- "Walk me through how you'd add logging to an existing object without modifying its class."
- "When would you choose Observer over a simple callback list?"
- "How do State and Strategy differ if their class diagrams look identical?"

Similar Problems

- **Command pattern** — related to Strategy in that it encapsulates an action as an object, but focuses on queuing/undoing operations rather than swapping algorithms.
- **Chain of Responsibility** — related to Decorator's wrapping structure, but each link decides whether to handle or pass along a request, not to add fixed behavior.
- **Factory Method** — often paired with Strategy, since something has to decide *which* strategy object to construct in the first place.
- **Pub/Sub systems (e.g., message queues)** — the distributed-systems version of Observer at scale.
- **Mediator pattern** — like Observer, but centralizes communication through a single mediator object instead of direct broadcasting.

Key Takeaways

- Patterns are best recognized by intent, not memorized by name.

- The single question "who's really in control?" separates Strategy from State and Decorator from Proxy — the two pairings that look identical on paper.
- Strategy: behavior handed in from outside. Observer: one object broadcasting to many. State: an object replacing itself from within. Decorator: wrapping to add behavior. Proxy: wrapping to control access.
- Real codebases don't always draw a hard line between Decorator and Proxy — what matters more is that your team agrees on what the wrapper is for.

Watch the Video

This entire breakdown started as a five-round challenge — if you want to test your own instincts against the clock before reading the reveals, watch the full video here: {LINK}

About the Series

This article is part of **Fun with Learning Technology**, a series built around one idea: understanding *how* something works beats memorizing *that* it works. Instead of listing pattern definitions, each episode puts you in the position of reading real code and reasoning your way to the answer — the same skill you'll actually use in a code review or an interview.

Call To Action

If this helped the patterns click, the video walks through all five rounds with the reveals timed for you to guess first — give it a watch and like it if it's useful. Subscribe on YouTube for more of these code-smell breakdowns, and subscribe to the Substack for deeper written dives that don't fit in a ten-minute video. Drop a comment with which round tricked you — Round 3 and Round 5 catch almost everyone.

Quick Review

5 quick reveals: Strategy, Observer, State, and 2 more — trigger?

Interchangeable algorithms=Strategy; publish/subscribe=Observer; behavior changes with internal mode=State.

Time complexity of identifying a pattern from a code snippet?

O(1) per round — fixed-size snippet, constant lookup against known pattern signatures.

Space complexity for a pattern-recognition round?

$O(1)$ — no extra data structures, just comparing snippet shape to memorized templates.

Trickiest bug: confusing State pattern with Strategy pattern?

Both swap behavior via composition; State changes itself internally on transitions, Strategy is set externally and stays fixed.

Observer vs Mediator — both involve multiple objects reacting to events?

Observer is one-to-many broadcast from a subject; Mediator centralizes communication between peers via one hub object.

Interview follow-up: how would you refactor a giant if/else chain choosing behavior?

Extract each branch into its own class implementing a common interface, then inject the chosen one — that's Strategy.

Speed challenge: name the design pattern, five rounds — free Python cheat sheet